

PENGENALAN BAHASA PEMROGRAMAN JAVA

Komponen bahasa pemrograman:

a. Identifier/variabel

Identifier : nama yang digunakan untuk memberi nama class, variabel, dan method
Variabel : tempat penyimpanan sementara di memori

Syarat penamaan identifier/variabel:

- Dapat terdiri dari huruf dan angka
- Harus diawali dengan huruf, underscore (_)
- Tidak boleh menggunakan simbol operator
- Tidak boleh ada spasi
- Tidak boleh menggunakan reserved word atau keyword yang dikenal java
- Bersifat *Case-Sensitive* (huruf besar dan kecil dibedakan)

Bentuk deklarasi variabel:

Tipe_data namaVariabel; atau
Tipe_data namaVariabel = nilai;

Contoh: int bilangan;
 bilangan =10;
 int bilangan =10;

 String nama;
 Nama ="....";
 String nama=".....";

b. Tipe data

Tipe data primitif adalah tipe data dasar. Tipe data ini terdiri dari 8 yang dikelompokkan menjadi 4 kelompok.

1. Integer, tipe bilangan bulat (byte, short, int, long)

Tipe	Ukuran	Jangkauan
byte	8 bit	-128 sampai 127
short	16 bit	-32,768 sampai 32,767
int	32 bit	-2,147,483,648 sampai 2,147,483,647
long	64 bit	-9,223,372,036,854,775,808 sampai +9,223,372,036,854,775,807

2. Floating point, tipe data bilangan pecahan (float, double)

Tipe	Ukuran	Range
float	32 bit	+/-1.4E-45 sampai +/-3.4028235E+38,
double	64 bit	+/-4.9E-324 sampai +/-1.7976931348623157E+308,

3. Karakter, terdiri dari sebuah karakter (*char*)
String merupakan kumpulan dari karakter. String bukan merupakan tipe data primitif, tapi sebuah class
4. Boolean, tipe data dengan nilai true/false (*boolean*)

Tipe data referensi merupakan data dari suatu objek (instance dari class).

Literal

Literal adalah suatu nilai konstan.Literal/nilai konstan disesuaikan dengan tipe data dari variabel yang menampungnya.

Literal terdiri dari:

- 1. **Literal integer.** Nilai literal integer merupakan bilangan bulat.
Literal integer dapat berupa bilangan bulat desimal (basis 10), oktal (basis 8), dan hexadesimal (basis 16).
Penulisan bilangan oktal diawali dengan tanda bilangan 0
Penulisan bilangan hexadesimal diawali dengan tanda bilangan 0x atau 0X
- 2. **Literal floating point,** nilai literal berupa bilangan pecahan.
Literal ini dapat ditulis menggunakan notasi biasa, menggunakan titik sebagai penanda pecahan.
Dan menggunakan notasi ilmiah (scientific notation), menggunakan lambang E (eksponensial)
- 3. **Literal Character (Char)**
Untuk data karakter dalam penulisan diapit tanda petik tunggal (‘).
Jika ada data yang tidak dapat dituliskan langsung, seperti karakter ganti baris, tabulasi, backspace, dll dapat digunakan escape sequences.Cara penulisannya menggunakan karakter \ (backslash).

Escape sequences

Escape sequence	Keterangan
\ddd	Karakter Oktal (ddd)
\uxxxx	Karakter hexadesimal (xxxx)
\’	Menghasilkan tanda ‘
\”	Menghasilkan tanda “
\\	Menghasilkan tanda \
\r	Carriage return
\n	Pindah baris
\f	Form feed
\t	Tabulasi
\b	Backspace

Untuk literal string harus diapit tanda kutip ganda (“)

4. Literal boolean

Untuk tipe data boolean hanya mengenal dua nilai yaitu **true** atau **false**

- c. Operator
Operator adalah simbol yang digunakan untuk pengoperasian terhadap operand.
Terdiri dari:
 - 1. **Operator penugasan,** pemberian nilai ke variabel
Contoh:
int nil=10;
int a,b,c;
a=b=c=5
 - 2. **Operator matematika (aritmatika),** digunakan dalam ekspresi matematika
*, / , % , + , -
 - 3. **Operator increamentatau decreament**
++, --
 - 4. **Operator relasional,** digunakan untuk membandingkan nilai
> , < , >= , <=, !=, ==
 - 5. **Operatorlogika,** untuk melakukan operasi pada operand yang bertipe data boolean

&& , || , == , !=

6. Operator ?: (if – then – else)

Operator ini digunakan jika ada opsi (pilihan) dalam proses

Bentuk umum:

(kondisi) ? Ekspresi_benar : ekspresi_salah

Contoh:

Nilai = (x > y)? x/10: x*20;

- d. Komentar program, merupakan keterangan program yang tidak akan diproses
Simbol yang digunakan:
// : untuk satu baris
/* */ : untuk beberapa baris (mengapit baris-baris komentar)
- e. Ekspresi, cara penulisan (ungkapan) yang melibatkan operator dan operand (penulisan rumus)
- f. Statement, pernyataan yang merupakan suatu perintah untuk melaksanakan tugas tertentu.

A. I/O Stream output

System.out.print() atau System.out.println()

Bentuk umum:

System.out.print/println(“teks”);→teks

System.out.print/println(namaVariabel);→nilai dari variabel

System.out.print/println(“teks” + namaVariabel);

Cout>>”hasil”>>hsl;

Cara penulisan aplikasi/program:

```
[modifier] class NamaClass {  
    public static void main (String[ ] args) {  
        .....  
        .....  
        .....  
    }  
}
```

} Pernyataan-pernyataan

Luas & Keliling segiempat
=====
Panjang = Cm
Lebar = cm
Luas = cm2 (panjang x lebar)
Keliling = cm (2 x panjang+lebar)
=====

Nama file diberi ekstensi .java (nama file harus sama dengan nama class)

Compile
Prompt> javac namafile.java

Running
Prompt> java namafile

Konversi suhu
=====
Celcius =(input)
Reamur =.....4/5xc
Fahrenheit =9/5xc+32

INPUT DATA DARI LUAR PROGRAM (Keyboard)

Menggunakan Class Scanner

Untuk membaca masukan dari keyboard, kita dapat menggunakan *class scanner* yang ada pada package /paket ***java.util.Scanner***. Metode-metode yang dapat digunakan untuk membaca masukan dari keyboard antara lain :

- a. **nextInt()** : untuk menerima tipe data integer
- b. **nextShort()** : untuk menerima tipe data short
- c. **nextLong()** : untuk menerima tipe data long
- d. **nextDouble()** : untuk menerima tipe data double
- e. **nextFloat()** : untuk menerima tipe data float
- f. **nextLine()/next()** : untuk menerima tipe data string
- g. **nextBoolean()** : untuk menerima tipe data boolean

Bentuk umum:

- memanggil/menggunakan class pada package/paket java
perintah : **import nama_paket.namaclass;**
I. contoh : **import java.util.Scanner;** → (ditulis sebelum public class)
- membentuk objek dari class input keyboard (dari luar program)
namaClass namaVar_objek = new namaClass(System.in);
contoh :

II. **Scanner input = new Scanner (System.in);**

- menggunakan metode untuk input data dari class **Scanner**
namaVar=nilai;
namaVar = namaVar_objek.metode; → cin>>namaVar;

contoh:

III. **nama = input.nextLine();**
alas=100; → cin>>alas;
alas=.....
bil=input.nextInt();
int bil;
bil=0

- a. **nextInt()** : untuk menerima tipe data integer
 - b. **nextShort()** : untuk menerima tipe data short
 - c. **nextLong()** : untuk menerima tipe data long
 - d. **nextDouble()** : untuk menerima tipe data double
 - e. **nextFloat()** : untuk menerima tipe data float
 - f. **nextLine()/next()** : untuk menerima tipe data string
 - g. **nextBoolean()** : untuk menerima tipe data boolean

Pengarah Tipe (Type-Casting)

Type Sumber	Type Tujuan
byte	short, char, int, long, float, double
short	int, long, float, double
char	int, long, float, double
int	long, float, double
long	float, double
float	double

Bentuk Umum:

Tipe_data namaVar = (*tipe_data_tujuan*) namaVar;

Contoh:

int i = 9;

float paksa = **(float)** i/2; Hasil → paksa = 4.5

contoh:

```
import java.util.Scanner;

public class ScannerInt{
    public static void main(String[] args){
        Scanner input = new Scanner(System.in);
        System.out.print("Ketik bilangan pertama : ");
        int bilangan1 = input.nextInt();
        System.out.print("Ketik bilangan kedua : ");
        int bilangan2 = input.nextInt();
        System.out.print("Hasil perkalian: " + (bilangan1 * bilangan2));
    }
}
```

Pernyataan berkondisi

Bentuk pernyataan berkondisi/bersyarat untuk pengambilan keputusan.

Pernyataan yang digunakan: if , switch - case

Bentuk Umum I:

```
if (kondisi/syarat)
{
    .....
    .....
}
else
{
    .....
    .....
}
```

Bentuk Umum II:

```
if (kondisi/syarat1)
{
.....
}
else if (kondisi/syarat2)
{
.....
}
.....
else if (kondisi/syaratn-1)
{
.....
}
else
{
.....
}
}
```

Bentuk Umum III

```
switch (nmVar)
{
case nilai-1:
.....
break;
case nilai-2:
.....
break;
case nilai-3:
.....
break;
default:
.....
}
}
```

Pengulangan (Looping)

Digunakan untuk melakukan proses yang memerlukan pengulangan (proses yang akan diulang beberapa kali)

Pernyataan : **for, while, do-while**

Bentuk Umum I:

```
for (nilai_awal; kondisi/syarat; step)
{
.....
}
}
```

Bentuk Umum II:

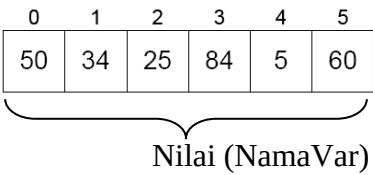
```
while (kondisi/syarat)
{
.....
}
}
```

Bentuk Umum III:

```
do
{
.....
} while (kondisi/syarat);
```

Array

Array adalah suatu tipe yang dibentuk dari tipe data primitif untuk menyimpan sejumlah item/nilai yang bertipe data sama. Array memungkinkan untuk menyimpan data maupun referensi obyek dalam jumlah banyak dan terindeks. Array menggunakan indeks integer untuk menentukan urutan elemen-elemennya, dimana elemen pertamanya dimulai dari indeks 0, elemen kedua memiliki indeks 1, dan seterusnya.



```
Nilai[0]=50;
Nilai[1]=34;
Nilai[2]=25;
Nilai[3]=84;
Nilai[4]=5;
Nilai[5]=60;
```

```
For(a=1;a<=6;a++){
Nilai[a]=inp.nextInt();
}
Cetak Nilai;
For(a=0;a<6;a++){
System.out.print (Nilai[a]);
}
```

Deklarasi variabel array 1 dimensi

```
tipe_data/class [ ] namaVar;           atau   tipe_data/class namaVar [ ];  
int [ ] Nilai;  
String [ ] nama;
```

Menyiapkan var array untuk menampung sejumlah data

```
namaVar = new tipe_data/class [jumlah_data];  
Nilai = new int[angka/namavar/jml_data];
```

```
Nilai=new int[6]           atau           Jml=6;  
Nilai=new int[Jml]
```

```
tipe_data[ ] namavar = new tipe_data[jml_data];
```

```
byte[ ] arr=new byte[6];  
int nilai[ ] = new int[10];
```

Mendeklarasikan sekaligus memberi nilai pada var array

```
Tipe_data/class [ ] namaVar = {nilai1, nilai2, nilai3, ...};  
int [ ] Nilai = {50, 34, 25,84,5, 60};
```

Passing Array

```
public static void main(String[] args) {  
    byte[] arr=new byte[3];  
    byte[] x=    calc(arr);  
        x = arr ( not null);  
}
```

```
public static byte[] calc (byte[] arr)  
{  
    // proses  
    return arr;  
}
```

Contoh:

```
public class CthArr {  
    public static void main(String[] args) {  
        byte[] arr = null;  
        byte[] x = calc(arr);  
  
        System.out.println(x.length);  
        for (int i=0;i<3;i++)  
            System.out.println(x[i]);  
    }  
}
```

```

public static byte[] calc(byte[] arr) {
    arr = new byte[3];
    arr[0]=34;
    arr[1]=30;
    arr[2]=4;

    return arr;
}

```

FORMAT ANGKA

Menggunakan package : **java.text.DecimalFormat**

I. **memanggil class pada package/paket (menggunakan class DecimalFormat)**

import java.text.DecimalFormat;

membentuk objek class DecimalFormat

classformat *namavarobj* = **new classformat**("format_angka");

format angka diwakili degan simbol : # , 0

contoh :

DecimalFormat formatku=new DecimalFormat ("#,###,##0.##")

System.out.println(formatku.format(bil));

II. **menggunakan objek class format**

namavarobj.metode(namaVar_angka);

formatku.format(bil)

contoh: 1. System.out.format("%d%n", bil); 2. System.out.format("%8.2f%n", bil);

Contoh1:

```
import java.text.DecimalFormat;
```

```
import java.text.DecimalFormatSymbols;
```

```
public class KurInd {
```

```
    public static void main(String[] args) {
```

```
        double harga = 1200000;
```

```
        DecimalFormat kursIndonesia = (DecimalFormat) DecimalFormat.getCurrencyInstance();
```

```
        DecimalFormatSymbols formatRp = new DecimalFormatSymbols();
```

```
        formatRp.setCurrencySymbol("Rp. ");
```

```
        formatRp.setMonetaryDecimalSeparator(',');
```

```
        formatRp.setGroupingSeparator('.');
```

```
        kursIndonesia.setDecimalFormatSymbols(formatRp);
```

```
        System.out.printf("Harga: %s %n", kursIndonesia.format(harga));
```

```
    }
```

```
}
```

Contoh 2:

```
totbayar=harga*bayar;
```

```
DecimalFormat f=new DecimalFormat("#.##");
```

```
System.out.print (f.format(totbayar));
```

Penanganan Eksepsi

Digunakan untuk mengatasi kesalahan pada saat program dieksekusi (bukan pada saat dikompilasi).

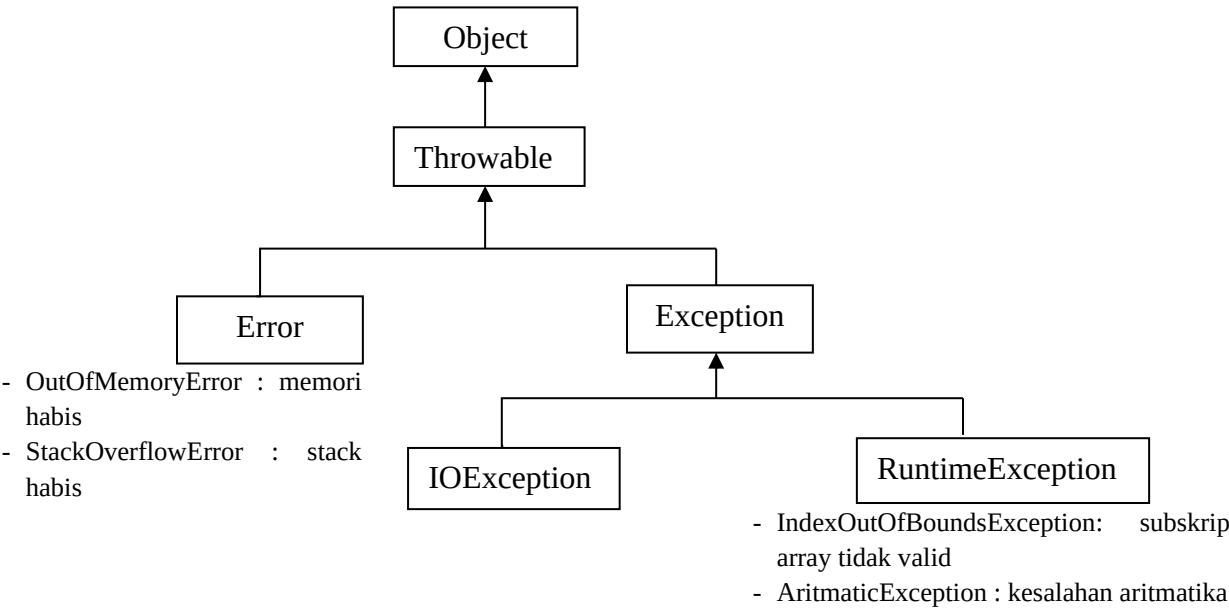
- Menangani kesalahan dengan menuliskan kode penanganan kesalahan.
- Mengetahui urutan kesalahan yang terjadi dalam sebuah metode dan semua metode yang terlibat dalam eksepsi.
- Program tidak akan berhenti jika terjadi eksepsi

Bentuk Umum:

```
try {  
    Blok pernyataan  
    .....  
    .....  
    int nilai=input.nextInt();  
}  
//catch (class parameter) {  
catch(Exception e) {
```

```
    Blok pernyataan yang dilakukan jika terjadi eksepsi  
    .....  
    System.out.print(“Salah memasukkan data”);  
    System.out.print(e.getMessage());  
}
```

Class yang berhubungan dengan Eksepsi



Pengenalan *Class*, *Object*, dan *Method*

Class

- *Class* adalah struktur dasar dari OOP (*Object Oriented Programming*).
- *Class* merupakan konsep pokok di bahasa pemrograman berorientasi obyek
- *Class* mendefinisikan bentuk dan perilaku obyek.
- *Class* terdiri dari dua tipe anggota yang disebut dengan *field* (attribut/properti) dan *method* (metode).

Field merupakan tipe data yang didefinisikan oleh *class*, sementara *method* merupakan operasi.

Pada pemrograman sebelumnya, *class* hanya mengkapsulkan metode (berbentuk fungsi) *main()* untuk menunjukkan sintaks dasar bahasa java.

Object

- Sebuah obyek adalah sebuah *instance* (keturunan) dari *class*.
- Obyek adalah berisi sekumpulan variabel dan *method* yang berhubungan (Setiap obyek dibangun dari sekumpulan data (atribut) yang disebut variabel/property).

Variabel menjabarkan karakteristik khusus dari obyek, dan juga terdiri dari sekumpulan *method* yang menjabarkan tingkah laku dari obyek.

Variabel dan *method* dalam obyek Java secara formal diketahui sebagai variabel *instance* dan *method instance*, dibedakan dari variabel *class* dan *method class*.

Method

- *Method* adalah bagian-bagian kode yang dapat dipanggil oleh program utama atau dari metode lainnya untuk menjalankan fungsi yang spesifik.

Karakteristik dari metode :

- a. Dapat mengembalikan satu nilai atau tidak sama sekali
- b. Dapat menerima beberapa parameter yang dibutuhkan atau tidak ada parameter sama sekali. Parameter bisa juga disebut sebagai argumen dari fungsi.
- c. Setelah metode selesai dieksekusi, dia akan kembali pada metode yang memanggilnya.

Membuat *Class*

Sebelum menulis/membuat *class*, perhatikan:

1. **Dimana** *class* akan digunakan dan **bagaimana** *class* tersebut akan digunakan.
2. **Nama** yang tepat dan tuliskan seluruh informasi atau properti yang ingin diisi pada *class*. Jangan sampai terlupa untuk menuliskan secara urut *method* yang akan digunakan dalam *class*.

Bentuk umum:

```
<modifier>class namaClass
{
    //deklarasi variabel
    .....
    //deklarasi method
    .....
}
```

Membentuk Method

Bentuk umum:

```
<modifier><returnType> namaMethod (parameter)
{
    .....
    .....
    return namaVar/rumus;
}
```

Ruang Lingkup Variabel

Variabel dipanggil berdasarkan lingkungannya, dimulai dari blok yang paling kecil, kemudian blok di luar blok tersebut.

1. **Local Variable** : digunakan di dalam method atau blok pernyataan yang lebih kecil dari itu
2. **Parameter** : variabel yg ada di dalam pernyataan (argument) method
3. **Instance Variable**: variabel yang memiliki nilai yang berbeda di setiap objek
4. **Class Variable** : variabel yang berlaku di suatu class dan seluruh instan dari class tersebut (objek).

Untuk memanggil variabel class dapat digunakan keyword ***this***

MODIFIER

Modifier adalah keyword yang diletakkan di depan class, interface, variable (field) atau method

Jenis Modifier:

- ✓ **Access Modifier** :Pengaturan pengaksesan dari instance variable dan method

Terdiri dari:

- **public** : pengaksesan suatu instance variabel/metode dapat dilakukan dari luar class
- **private** : pengaksesan suatu instance variabel/metode hanya dapat dilakukan di dalam class saja.
- **Tanpa Tanda** : pengaksesan suatu instance variabel dan method dapat dilakukan di dalam kelas dan kelas lain dalam satu paket

- ✓ **Static Modifier:**
 - Membuat method dan variable menjadi milik class, bukan object
 - Tidak perlu membuat object untuk penggunaan variable (field) dan method
- ✓ **Final Modifier** :Menyatakan sesuatu (class, method, variable) sudah final dan tidak adapat diubah

Modifier	Dalam Class yang Sama	Dalam Package yang Sama	Dalam SubClass	Dalam Package Lain
private	✓			
tanpa tanda	✓	✓		
protected	✓	✓	✓	
public	✓	✓	✓	✓

Pengarah Tipe (Type-Casting)

Tipe Sumber	Tipe Tujuan
byte	short, char, int, long, float, double
short	int, long, float, double
char	int, long, float, double
int	long, float, double
long	float, double
float	double

Bentuk Umum:

Tipe_data namaVar = (*tipe**data_tujuan*) namaVar;

Contoh:

double i = 10.56;
int paksa = (int) i; Hasil → paksa = 10

konversi data **String** ke tipe lain (angka)

kelas yang dapat digunakan : **Integer, Float, Double**

Integer.valueOf(data_String).intValue()
Double.valueOf(data_String).doubleValue()
Float.valueOf(data_String).floatValue()

Integer.parseInt(data_String)
Double.parseDouble(data_String)
Float.parseFloat(data_string)

konversi data angka ke String

Integer.toString (data_angka)
Double.toString (data_angka)

Konstruktor

Constructor adalah *method* dimana seluruh inisialisasi *object* ditempatkan.

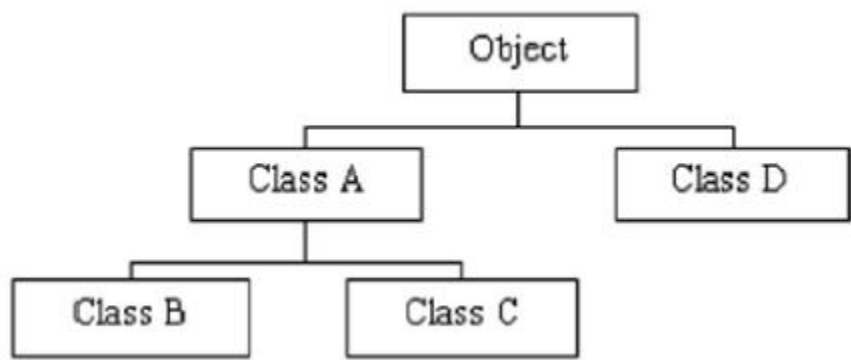
Berikut ini adalah *property* dari *constructor* :

- a. *Constructor* memiliki **nama yang sama dengan class**
- b. Sebuah *constructor* mirip dengan *method* pada umumnya, namun hanya informasi – informasi berikut yang dapat ditempatkan pada *header* sebuah *constructor*, *scope* atau identifikasi pengaksesan (misal: *public*), nama dari konstuktor dan parameter.
- c. *Constructor* tidak memiliki *return value*
- d. *Constructor* tidak dapat dipanggil secara langsung, namun harus dipanggil dengan menggunakan operator *new* pada pembentukan sebuah objek.

```
<modifier><className> (<parameter>)  
  
{  
    .....  
    .....  
}
```

Pewarisan

Dalam Java, semua *class*, termasuk *class* yang membangun Java API, adalah *subclasses* dari *superclass* *Object*. Contoh hirarki *class* diperlihatkan pada gambar 5.1 dibawah ini.Beberapa *class* di atas *class* utama dalam hirarki *class* dikenal sebagai *superclass*.Sementara beberapa *class* di bawah *class* pokok dalam hirarki *class* dikenal sebagai *subclass* dari *class* tersebut.



Membuat Class Pewarisan

```
<modifier> class subClass_pewaris extends superClass{

//konstruktor
.....
.....

//metode
.....
.....

}
```

Memanggil konstruktor SuperClass

```
super (arg1, arg2, ...);
```

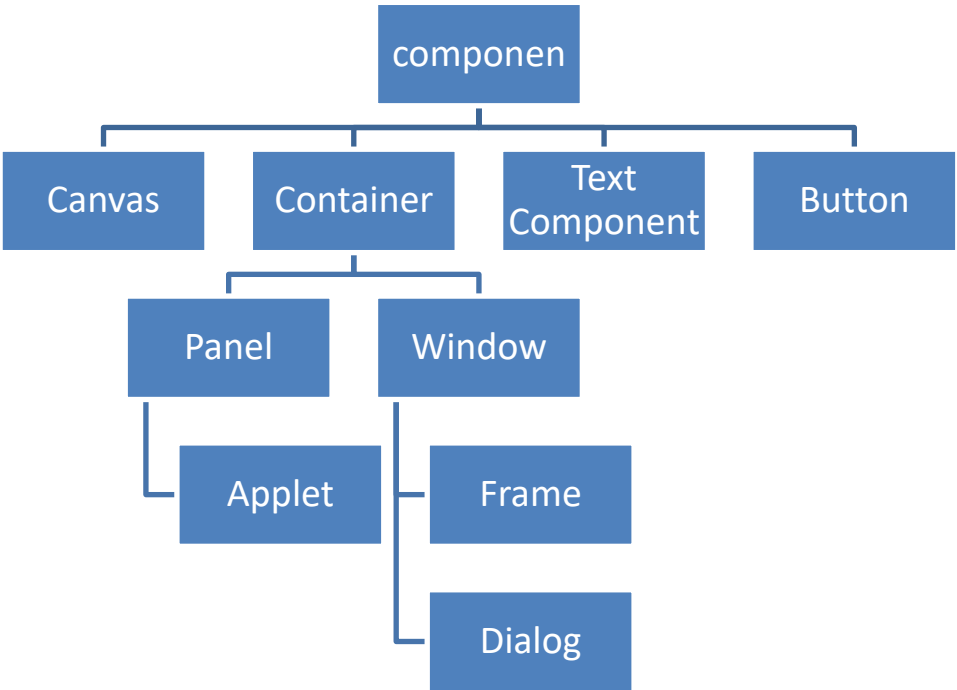
Graphical User Interface (GUI)

Java menyediakan API untuk membuat aplikasi berbasis grafis atau yang lebih dikenal dengan **Graphical User Interface** (GUI). Ada dua package untuk membuat GUI yaitu **AWT** (**Abstract Windowing Toolkit**) dan **Swing**. Dengan package ini dapat dibuat sebuah aplikasi grafis yang sangat menarik.

Beberapa fasilitas untuk keperluan aplikasi grafis yaitu :

- Pengaturan tata letak (**layout management**) komponen.
- **Event handling**, yaitu pendeteksian setiap event dan penentuan respon yang akan diberikan ketika pengguna menekan akan mengklik komponen tersebut untuk kemudian melakukan proses yang diinginkan.
- Manipulasi grafis dari komponen, meliputi teks, warna, ukuran, dan lain-lain.

AWT memiliki beberapa komponen untuk aplikasi GUI yaitu **button**, **canvas**, **checkbox**, **choice**, **label**, **list**, **scrollbar**, **textfield**, dan **textarea**. Setiap komponen ini memiliki fungsi dan cara pembuatan yang berbeda-beda.



Untuk pemrograman GUI menggunakan paket Swing
`import javax.swing.*;`

Membuat rancangan interface Menggunakan Class

Bentuk Umum:

```
namaClass namaVarobj = new namaClass();  
    atau  
namaClass namaVarObj;  
namaVarObj=new namaClass();
```

Frame menggunakan pewarisan dari **Class JFrame** dan **Konstruktor**

Pewarisan menggunakan : **extends**

Langkah membentuk objek/komponen GUI

1. Deklarasi
2. Membentuk objek
3. Mengatur posisi objek
4. Menampilkan/menambahkan objek pada komponen panel
5. Menampilkan panel dan frame
6. Memberi aksi (actionlistener) pada objek
7. Memberikan proses pada event

Contoh:

```
import javax.swing.*;  
import java.awt.*;  
import java.awt.event.*;  
public class TampilFrame1 extends JFrame  
{  
    JLabel nama, gaji, hasil;  
    JTextField namaField, gajiField, hslField;  
    JPanel panel, p2;  
    JButton btn, btn1;  
  
    public static void main(String args[])  
    {  
        TampilFrame1 frameku = new TampilFrame1();  
    }  
    public TampilFrame1()  
    {  
        super("Form Utama");  
        setSize(400,400);  
        setDefaultCloseOperation(EXIT_ON_CLOSE);  
        //setLocation(40,70); //(x,y) posisi  
        setLocationRelativeTo(null); //center  
        setResizable(true);  
  
        nama=new JLabel("Nama Karyawan ");  
        namaField=new JTextField("",30);  
        btn1=new JButton("Proses");  
        panel=new JPanel();  
        panel.setLayout(new GridLayout(3,2,0,2));  
        panel.add(nama);  
        panel.add(namaField);  
  
        //btn.setBounds(150,100, 20, 20);  
  
        getContentPane().add("North",panel);  
  
        btn1.addActionListener(new TampilFrame1.proses());  
  
        setVisible(true);  
    }  
    //tunjangan anak=jumlah anak x 20% x gaji pokok
```

```
class proses implements ActionListener {

    public void actionPerformed(ActionEvent e){
        JButton tombol = (JButton)e.getSource();
        //String s=e.getActionCommand();
        //if (s.equals("Proses") ){
        if (tombol.getText().equals("Proses") ){
            int nil1=Integer.parseInt(namaField.getText());
            int nil2=Integer.parseInt(gajiField.getText());
            int hs=nil1+nil2;
            hslField.setText(Integer.toString(hs));
        }
    }
}
}
```